LeetCode Interview Cheatsheet

Arrays, Strings & HashMap Patterns - SDE1 Prep

1. Product of Array Except Self

Problem

Return array where `answer[i]` = product of all elements except `nums[i]`. **No division allowed!**

Core Concept

Prefix & Suffix Products: For each position, multiply left products × right products.

Key Insight

- Use output array to store prefix products (space optimization!)
- Use single variable for suffix (scan right-to-left)
- Two passes: build prefix, then multiply by suffix

Approach

1. **Pass 1**: Build prefix products in answer array
2. **Pass 2**: Multiply by suffix products (right-to-left)

Code Skeleton

```
int[] answer = new int[n];
answer[0] = 1;

// Build prefix in answer
for (int i = 1; i < n; i++) {
    answer[i] = nums[i-1] * answer[i-1];
}

// Multiply by suffix
int suffix = 1;
for (int i = n-1; i >= 0; i--) {
    answer[i] *= suffix;
    suffix *= nums[i];
}
return answer;
```

Complexity

Time & Space

Time: O(n) - Two single passes
Space: O(1) - Only output array (doesn't count)

Edge Cases

- Array with zeros
- Array length = 2
- Negative numbers

2. Maximum Subarray (Kadane's)

Problem

Find contiguous subarray with largest sum.

Core Concept

Kadane's Algorithm: At each position, decide: extend current subarray OR start fresh?

Key Insight

- If previous sum is negative, it hurts future sums → **abandon it!**
- Greedy local choice = globally optimal
- Don't need actual subarray, just track best sum

The Decision

```
sum = Math.max(nums[i], sum + nums[i]);
        |           |
    start fresh   extend
```

Code Skeleton

```
int sum = nums[0];
int maxSum = nums[0];

for (int i = 1; i < n; i++) {
    sum = Math.max(nums[i], sum + nums[i]);
    maxSum = Math.max(maxSum, sum);
}
return maxSum;
```

Complexity

Time & Space

Time: O(n) - Single pass
Space: O(1) - Only two variables

Why Brilliant?

Reduces $O(n^2)$ problem to O(n) by realizing: negative prefix never helps!

3. Maximum Product Subarray

Problem

Find contiguous subarray with largest product.

Core Concept

Track Both Extremes: Unlike sum, negative × negative = positive!

Key Insight

- Large negative could become large positive!
- Track `max` (best product) AND `min` (worst product)
- When you hit negative, min and max swap roles
- Need `temp` to save old max before updating

Three Choices

```
max = Math.max(num*max, num*min, num);
            |         |         |
        extend    flip!    start fresh

min = Math.min(temp, num*min, num);
```

Code Skeleton

```
int min = 1, max = 1;
int globalMax = nums[0];

for (int num : nums) {
    int temp = num * max; // Save!

    max = Math.max(
        Math.max(num*max, num*min), num);
    min = Math.min(
        Math.min(temp, num*min), num);

    globalMax = Math.max(globalMax, max);
}
return globalMax;
```

Complexity

Time & Space

Time: O(n) - Single pass
Space: O(1) - Three variables

Why temp?

Without it, min calculation would use *updated* max value (wrong!)

4. Group Anagrams

Problem

Group strings that are anagrams of each other.

Core Concept

Canonical Form: Anagrams have same sorted string or same character count.

Key Insight

- Use sorted string as HashMap key
- All anagrams map to same key
- Alternative: character count array as key (faster!)

Two Approaches

Approach 1: Sorting (simpler)

```
Map<String, List<String>> map;
for (String s : strs) {
    char[] chars = s.toCharArray();
    Arrays.sort(chars);
    String key = new String(chars);

    map.putIfAbsent(key, new ArrayList<>());
    map.get(key).add(s);
}
return new ArrayList<>(map.values());
```

Approach 2: Counting (faster)

```
int[] count = new int[26];
for (char c : s.toCharArray()) {
    count[c - 'a']++;
}
String key = Arrays.toString(count);
```

Complexity

Sorting Approach

Time: O(n × m log m)
- n = # strings, m = max length
Space: O(n × m) - HashMap keys

Counting Approach

Time: O(n × m) - No sorting!
Space: O(n × m) - Same

Adaptations

- `int[52]` for upper+lowercase
- `int[128]` for all ASCII
- Convert to lowercase for case-insensitive

5. Top K Frequent Elements

Problem

Return k most frequent elements from array.

Core Concept

Min Heap of Size K: Keep smallest at top, remove when size > k.

Key Insight

- Min heap maintains k largest by removing smallest
- Heap size capped at k → O(log k) not O(log m)!
- When size > k, poll (remove minimum frequency)

Why Min Heap?

```
Freq: [3, 2, 1], k=2
Add (3): heap = [3]
Add (2): heap = [2,3]  (min at root)
Add (1): heap = [1,2,3], size>k!
  Poll(): remove 1, heap = [2,3]
Result: Top 2 frequencies!
```

Code Skeleton

```
// Step 1: Count frequencies
Map<Integer, Integer> map;
for (int num : nums) {
    map.put(num, map.getOrDefault(num,0)+1);
}

// Step 2: Min heap of size k
PriorityQueue<Map.Entry<Integer,Integer>>
    minHeap = new PriorityQueue<>(
        (a,b) -> a.getValue() - b.getValue()
    );

for (Map.Entry<Integer,Integer> e :
     map.entrySet()) {
    minHeap.offer(e);
    if (minHeap.size() > k) {
        minHeap.poll(); // Remove min
    }
}

// Step 3: Extract
int[] result = new int[k];
for (int i = 0; i < k; i++) {
    result[i] = minHeap.poll().getKey();
}
return result;
```

Complexity

Time & Space

Time: O(n + m log k)
- n for counting, m for heap ops
- **NOT** O(m log m)! Heap size = k
Space: O(m + k) - Map + heap

Alternative: Bucket Sort

O(n) time but more complex - skip for SDE1 unless time permits.

6. Subarray Sum Equals K

Problem

Count subarrays with sum = k.

Core Concept

Prefix Sum + HashMap: If `prefixSum[j] - prefixSum[i] = k`, then subarray from i+1 to j has sum k.

Key Insight

- Rearrange: `prefixSum[i] = prefixSum[j] - k`
- At position j, look for `(currentSum - k)` in map
- Store *frequency* not just existence (duplicates!)
- `map.put(0, 1)` handles subarrays from index 0

The Math

```
If currentSum = 10 and k = 3:
  Look for sum = 7 in map
  If found 2 times, add 2 to count
  (2 previous positions had sum=7)
```

Code Skeleton

```
Map<Integer, Integer> map;
map.put(0, 1); // CRITICAL!

int sum = 0, count = 0;
for (int num : nums) {
    sum += num;

    // Check BEFORE updating
    if (map.containsKey(sum - k)) {
        count += map.get(sum - k);
    }

    // Then update
    map.put(sum,
        map.getOrDefault(sum, 0) + 1);
}
return count;
```

Complexity

Time & Space

Time: O(n) - Single pass
Space: O(n) - HashMap worst case

Why map.put(0, 1)?

Handles subarrays starting from index 0. Without it, `nums=[3], k=3` would return 0 instead of 1!

Order Matters!

Check *before* updating map, otherwise current element would match itself!

7. Longest Substring No Repeat

Problem

Find length of longest substring without repeating characters.

Core Concept

Sliding Window + HashMap: Expand right, contract left when duplicate found.

Key Insight

- HashMap stores: char → last seen index
- When duplicate found, jump left pointer past it
- `Math.max(left, map.get(c)+1)` prevents moving backward!
- Window invariant: all chars in [left, right] are unique

Why Math.max?

```
s = "abba"
At 'a' (index 3):
  'a' was at index 0 (outside window)
  Without Math.max: left = 0+1 = 1
    Would move backward! Wrong!
  With Math.max: left = max(2, 1) = 2
    Stays in place! Correct!
```

Code Skeleton

```
Map<Character, Integer> map;
int left = 0, maxLen = 0;

for (int right = 0; right < n; right++) {
    char c = s.charAt(right);

    if (map.containsKey(c)) {
        // Jump past duplicate
        left = Math.max(left,
                        map.get(c) + 1);
    }

    map.put(c, right); // Update index

    maxLen = Math.max(maxLen,
                      right - left + 1);
}
return maxLen;
```

Complexity

Time & Space

Time: O(n) - Each char visited once
Space: O(k) - k = unique chars
- O(1) for fixed charset (26, 128)
- O(n) worst case (all unique)

Window Length

Always: `right - left + 1` (inclusive of both ends)

Pattern Recognition

Prefix/Suffix Products

When: Need value without current element
How: Build prefix left-to-right, suffix right-to-left
Examples: Product Except Self

Kadane's Pattern

When: Max/min subarray sum/product
How: Track current best, decide extend vs restart
Examples: Max Subarray, Max Product

HashMap for Anagrams

When: Group by characteristic
How: Canonical form (sorted/count) as key
Examples: Group Anagrams

Min/Max Heap for Top K

When: Find k largest/smallest
How: Min heap of size k for k largest
Examples: Top K Frequent

Prefix Sum + HashMap

When: Count subarrays with property
How: Store prefix sums, look for difference
Examples: Subarray Sum = K, Contiguous Array

Sliding Window

When: Substring/subarray with constraint
How: Two pointers, expand/contract
Examples: Longest Substring No Repeat

Common Pitfalls

Off-by-One Errors

- Window length: `right - left + 1` (not just `right - left`)
- Array bounds: `i < n` not `i <= n`
- Heap operations: size $>$ k not size $\geq$ k

HashMap Operations

- Use `getOrDefault(key, 0)` for counting
- Check existence before get: `containsKey()`
- Update after checking (subarray sum pattern)

Initialization

- `map.put(0, 1)` for prefix sum problems
- Initialize to `nums[0]` not 0 (Kadane's)
- Set proper comparators for PriorityQueue

State Management

- Save old value with `temp` before updating (max product)
- Use `Math.max(left, ...)` to prevent backward movement
- Track both min and max when negatives matter

Complexity Quick Reference

Data Structure	Operations
HashMap	get/put: O(1) avg
TreeMap	get/put: O(log n)
PriorityQueue	offer/poll: O(log n)
	peek: O(1)
Arrays.sort()	O(n log n)
Pattern	**Complexity**
Single pass	O(n)
Nested loops	$O(n^2)$
Sorting	O(n log n)
Binary search	O(log n)
Heap of size k	O(n log k)

Space Complexity Notes

- Output array usually doesn't count
- HashMap: O(k) where k = unique elements
- Fixed-size array (26, 128): O(1)
- Recursion: O(h) stack space, h = depth

Interview Strategy

Problem-Solving Steps

1. **Clarify**: Ask about edge cases, constraints
2. **Example**: Walk through example
3. **Brute Force**: Mention briefly (30 sec)
4. **Optimize**: Identify pattern, explain approach
5. **Code**: Write clean, explain as you go
6. **Test**: Walk through example, edge cases
7. **Analyze**: Time and space complexity

Communication Tips

- Think out loud
- Explain trade-offs
- Ask for hints if stuck (don't stay silent)
- Start with working solution, then optimize

Time Management (45 min)

- 0-5 min: Understand problem
- 5-8 min: Discuss approach
- 8-30 min: Code solution
- 30-40 min: Test and debug
- 40-45 min: Complexity and follow-up

Edge Cases Checklist

- Empty array/string: `n = 0`
- Single element: `n = 1`
- Two elements: `n = 2`
- All same elements
- All unique elements
- Negative numbers
- Zeros in array
- k = 1 or k = n
- Duplicates
- Very large numbers (overflow?)

Java Quick Reference

Array

```
int[] arr = new int[n];
Arrays.sort(arr);
Arrays.toString(arr);
arr.length
```

String

```
s.length()
s.charAt(i)
s.toCharArray()
s.substring(start, end)
```

HashMap

```
Map<K,V> map = new HashMap<>();
map.put(key, value)
map.get(key)
map.containsKey(key)
map.getOrDefault(key, defaultVal)
map.putIfAbsent(key, value)
map.entrySet()
map.keySet()
map.values()
```

PriorityQueue

```
// Min heap (default)
PriorityQueue<Integer> minHeap =
    new PriorityQueue<>();

// Max heap
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a,b) -> b - a);

// Custom comparator
PriorityQueue<Map.Entry<K,V>> pq =
    new PriorityQueue<>(
        (a,b) -> a.getValue() - b.getValue()
    );

pq.offer(element)  // add
pq.poll()          // remove top
pq.peek()          // view top
pq.size()
```

Math

```
Math.max(a, b)
Math.min(a, b)
Math.abs(x)
```

Pro Tips:

- Draw examples before coding
- Use meaningful variable names
- Check for integer overflow with large products
- HashMap beats nested loops in most cases
- When stuck, start with brute force
- Practice explaining your thought process

Good luck with your interviews!